

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

————— * —————



BÁO CÁO ĐỒ ÁN MÔN HỌC
HỆ THỐNG NHÚNG

Đề tài:

HỆ THỐNG GIÁM SÁT
NHIỆT ĐỘ THỜI GIAN THỰC

Sử dụng STM32F405 và FreeRTOS

Nhóm thực hiện: Nhóm 2

Thành viên:	23200075	Phan Thành Đạt
	23200020	Phan Vũ Đăng Khoa
	23200129	Chu Hoàng Quý
	23200163	Võ Trần Mạnh Tuấn
	21200262	Nguyễn Đức Yên

Lời Cảm Ơn

Chúng em xin chân thành cảm ơn thầy giảng dạy môn Hệ thống nhúng đã tận tình hướng dẫn, truyền đạt kiến thức và tạo điều kiện để nhóm chúng em hoàn thành đồ án này.

Trong quá trình thực hiện, chúng em đã có cơ hội áp dụng những kiến thức lý thuyết về vi điều khiển ARM, Real-Time Operating System (RTOS), giao tiếp ngoại vi (SPI, SDIO, ADC) vào một hệ thống nhúng hoàn chỉnh. Đồ án không chỉ giúp chúng em củng cố kiến thức chuyên môn mà còn rèn luyện kỹ năng làm việc nhóm, tư duy thiết kế hệ thống và khả năng giải quyết vấn đề thực tế.

Mặc dù đã cố gắng hết sức, báo cáo khó tránh khỏi thiếu sót. Chúng em rất mong nhận được sự góp ý của thầy để hoàn thiện hơn.

Nhóm 2

Tóm Tắt Đề Tài

Đồ án tập trung xây dựng hệ thống giám sát nhiệt độ thời gian thực trên nền tảng vi điều khiển STM32F405RGT6 kết hợp với hệ điều hành thời gian thực FreeRTOS. Hệ thống thực hiện thu thập dữ liệu từ cảm biến nhiệt độ nội bộ và kênh ADC ngoại vi một cách liên tục, xử lý lọc nhiễu bằng thuật toán EMA và hiển thị trực quan lên màn hình LCD 2.8 inch. Điểm nổi bật của đề tài là việc áp dụng kiến trúc đa nhiệm để xử lý đồng thời các tác vụ hiển thị, tương tác cảm ứng và ghi dữ liệu lịch sử vào thẻ nhớ SD Card theo định dạng FAT32. Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, độ trễ thấp và đáp ứng tốt các yêu cầu về tính thời gian thực trong các ứng dụng nhúng.

Danh Mục Từ Viết Tắt

Viết tắt	Ý nghĩa đầy đủ
ADC	Analog-to-Digital Converter
RTOS	Real-Time Operating System
SPI	Serial Peripheral Interface
SDIO	Secure Digital Input Output
FAT32	File Allocation Table 32
EMA	Exponential Moving Average
MCU	Microcontroller Unit
RTC	Real-Time Clock
HAL	Hardware Abstraction Layer
CMSIS	Cortex Microcontroller Software Interface Standard
DMA	Direct Memory Access
LCD	Liquid Crystal Display

Mục lục

Lời Cảm Ơn	1
Tóm Tắt Đề Tài	2
Danh Mục Từ Viết Tắt	3
1 Giới Thiệu	8
1.1 Đặt vấn đề	8
1.2 Mục tiêu đề án	8
1.3 Phạm vi và giới hạn	9
1.3.1 Phần cứng sử dụng	9
1.3.2 Phần mềm và công cụ	9
1.3.3 Giới hạn	9
1.4 Yêu cầu sử dụng RTOS trong đề án	9
1.5 Cấu trúc báo cáo	10
2 Thiết Kế Phần Cứng	11
2.1 Tổng quan hệ thống	11
2.2 Sơ đồ khối hệ thống	11
2.3 Vi điều khiển STM32F405RGT6	11
2.4 Board phát triển Open405R-C	13
2.5 Màn hình LCD 2.8” cảm ứng điện trở	14
2.5.1 Thông số LCD	15
2.5.2 Thông số Touch Panel	15
2.5.3 Sơ đồ kết nối chân	15
2.6 Module SD Card	16
2.7 ADC ngoại	16
2.8 Cấu hình Clock	17
2.8.1 PLL Configuration	17
2.8.2 Clock Tree	17
2.8.3 Cấu hình nguồn clock trong code	18
2.9 Sơ đồ kết nối tổng thể	18
3 Thiết Kế Phần Mềm	20
3.1 Tổng quan kiến trúc phần mềm	20
3.1.1 Cấu trúc thư mục project	20
3.2 Kiến trúc FreeRTOS	21
3.2.1 Cấu hình FreeRTOS Heap	21
3.3 Sơ đồ tương tác giữa các Task	22

3.4	Cơ chế đồng bộ hóa	22
3.4.1	Mutex SPI (MutexSPI)	22
3.4.2	Thread Notification Flags	23
3.4.3	Biến chia sẻ volatile	23
3.5	Cấu hình ADC	24
3.5.1	Công thức tính nhiệt độ	25
3.6	Cấu hình RTC	26
3.6.1	Tự động đồng bộ thời gian từ thời gian biên dịch	26
3.7	Cấu hình FatFs	27
3.8	Driver LCD và Touch Panel	27
3.8.1	Thư viện LCD28	27
3.8.2	Các hàm LCD chính	28
3.9	Cấu trúc dữ liệu xem dữ liệu	28
4	Triển Khai Chi Tiết	30
4.1	TaskSensor	30
4.2	TaskGUI — Giao diện và xử lý cảm ứng	31
4.2.1	Thiết kế giao diện	31
4.2.2	Giao diện chính	31
4.2.3	Kỹ thuật vẽ Gradient trên LCD	32
4.2.4	Xử lý cảm ứng	33
4.2.5	Chế độ Data Viewer	33
4.2.6	Cơ chế Smart Redraw	34
4.3	TaskSD — Quản lý SD Card và ghi dữ liệu	35
4.3.1	Quy trình khởi tạo SD Card	35
4.3.2	Override BSP_SD_Init	36
4.3.3	Ghi dữ liệu mỗi 3 giây (dựa trên RTC)	37
4.3.4	Đọc file cho chế độ xem dữ liệu	37
4.3.5	Xóa dữ liệu	38
4.4	Mã lỗi SD Card	38
5	Kết Quả Thực Nghiệm	39
5.1	Kết quả hiển thị LCD	39
5.1.1	Màn hình chính	39
5.1.2	Màn hình khi đang ghi dữ liệu	40
5.1.3	Màn hình xem dữ liệu	41
5.2	Kết quả dữ liệu trên SD Card	42
5.3	Phân tích hiệu năng RTOS	43
5.4	Kiểm thử chức năng	43
6	Kết Luận	44
6.1	Kết quả đạt được	44
6.2	Các kỹ thuật nổi bật	44
6.3	Hạn chế	45
6.4	Hướng phát triển	45

Danh sách hình vẽ

2.1	Sơ đồ khối hệ thống	11
2.2	Board Open405R-C thực tế	12
2.3	Sơ đồ chân STM32F405RGT6	13
2.4	Board Open405R-C kết nối với LCD 2.8" và SD Card	14
2.5	Sơ đồ kết nối chân LCD Module với board Open405R-C	16
2.6	Sơ đồ Clock Tree hệ thống	18
2.7	Sơ đồ nối dây tổng thể của hệ thống theo cấu hình <code>doan.ioc</code>	18
3.1	Cấu trúc project trong STM32CubeIDE	20
3.2	Cấu hình FreeRTOS Tasks trong CubeMX	22
3.3	Sơ đồ tương tác giữa các Task và tài nguyên hệ thống	22
3.4	Cấu hình ADC trong CubeMX	25
4.1	Giao diện Home (trái) và khi đang PLAY (phải)	32
4.2	Giao diện Data Viewer trên LCD	34
4.3	Flowchart quy trình khởi tạo SD Card	36
5.1	Kết quả hiển thị giao diện Home trên LCD	39
5.2	Kết quả hiển thị LCD khi đang ở chế độ PLAY	40
5.3	Kết quả hiển thị giao diện xem dữ liệu thực tế trên LCD	41
5.4	Nội dung file <code>data.txt</code> mở trên máy tính	42

Danh sách bảng

1.1	So sánh Super Loop và RTOS	10
2.1	Thông số chính STM32F405RGT6	12
2.2	Thông số module LCD 2.8”	15
2.3	Thông số Touch Panel	15
2.4	Bảng kết nối chân LCD và Touch	15
2.5	Cấu hình SDIO	16
2.6	Bảng tần số các bus	17
2.7	Tổng hợp chân GPIO sử dụng trong đồ án	19
3.1	Danh sách Task trong hệ thống	21
3.2	Cấu hình bộ nhớ FreeRTOS	21
3.3	Cấu hình kênh ADC	24
3.4	Cấu hình FatFs	27
3.5	So sánh driver gốc và driver đã điều chỉnh	28
4.1	Bố cục giao diện LCD chính	31
4.2	Bảng mã lỗi SD Card hiển thị trên LCD	38
5.1	Thống kê thời gian thực thi các Task	43
5.2	Bảng kiểm thử chức năng hệ thống	43

Chương 1

Giới Thiệu

1.1 Đặt vấn đề

Trong lĩnh vực hệ thống nhúng, việc giám sát và thu thập dữ liệu cảm biến theo thời gian thực là yêu cầu phổ biến trong công nghiệp, y tế và nghiên cứu khoa học. Một hệ thống giám sát hoàn chỉnh không chỉ cần đọc dữ liệu từ cảm biến mà còn phải hiển thị thông tin trực quan, lưu trữ dữ liệu và cho phép người dùng tương tác thông qua giao diện — tất cả các tác vụ này phải diễn ra đồng thời mà không được phép làm gián đoạn lẫn nhau.

Kiến trúc Super Loop truyền thống — phương pháp phổ biến nhất khi mới tiếp cận vi điều khiển — gặp nhiều hạn chế khi xử lý nhiều tác vụ có mức ưu tiên khác nhau. Ví dụ: nếu tác vụ ghi SD Card mất 100ms để hoàn thành, toàn bộ hệ thống sẽ bị “đóng băng” trong suốt thời gian đó, dẫn đến màn hình LCD bị “đơ”, cảm ứng không phản hồi, và cảm biến bị bỏ sót mẫu đo.

RTOS giải quyết triệt để vấn đề này bằng cách chia chương trình thành các *task* độc lập, được bộ lập lịch (scheduler) quản lý tự động theo mức ưu tiên. Mỗi task hoạt động như một “chương trình con” riêng biệt, có stack riêng, có thể bị ngắt (preempt) bất cứ lúc nào nếu có task ưu tiên cao hơn cần chạy.

1.2 Mục tiêu đề án

Đề án xây dựng hệ thống nhúng trên vi điều khiển **STM32F405RGT6** (board Open405R-C) với FreeRTOS, thực hiện **song song** các chức năng sau:

- 1. Đọc nhiệt độ liên tục:** Sử dụng cảm biến nhiệt độ nội bộ của chip STM32, kết hợp bù trừ sụt áp nguồn (VREFINT) và bộ lọc EMA để đạt độ chính xác cao. Giá trị được hiển thị lên màn hình LCD 2.8” cảm ứng điện trở với giao diện đồ họa trực quan.
- 2. Đọc ADC ngoại:** Ngoài nhiệt độ nội, hệ thống còn đọc thêm kênh ADC_IN10 (PC0) kết nối với biến trở hoặc cảm biến bên ngoài, cho phép mở rộng khả năng đo lường.
- 3. Ghi dữ liệu:** Khi người dùng chạm nút “PLAY” trên LCD, cứ mỗi 3 giây hệ thống ghi một bản ghi gồm {Thời gian RTC, Nhiệt độ, Giá trị ADC chip, Giá trị ADC analog} vào file `data.txt` trên thẻ nhớ SD Card.

4. **Xem dữ liệu:** Khi chạm nút “PAUSE”, hệ thống dừng ghi và chuyển sang chế độ xem dữ liệu — hiển thị toàn bộ lịch sử dữ liệu đã lưu trên màn hình, hỗ trợ phân trang Trước/Sau, hiển thị dữ liệu mới nhất lên đầu, và cho phép xóa toàn bộ dữ liệu.

1.3 Phạm vi và giới hạn

1.3.1 Phần cứng sử dụng

- Board phát triển: **Open405R-C** (Waveshare), MCU STM32F405RGT6.
- LCD: Module 2.8” ILI9341, giao tiếp SPI, độ phân giải 240×320 pixel.
- Touch Panel: Cảm ứng điện trở, IC driver XPT2046, chia sẻ bus SPI1 với LCD.
- SD Card: Giao tiếp qua SDIO hỗ trợ cả 4-bit và 1-bit, DMA transfer.
- Thạch anh ngoại: HSE 25 MHz làm clock hệ thống, LSE 32.768 kHz cho RTC.

1.3.2 Phần mềm và công cụ

- IDE: **STM32CubeIDE** phiên bản 1.17+.
- Cấu hình ngoại vi: **STM32CubeMX** được tích hợp sẵn trong IDE.
- RTOS: **FreeRTOS** thông qua CMSIS-RTOS v2 API.
- Hệ thống file: **FatFs** hỗ trợ định dạng FAT32.
- Thư viện HAL: STM32Cube FW_F4 V1.28.3.
- Driver LCD: Thư viện tùy chỉnh dựa trên bộ demo 2.8” của Waveshare.

1.3.3 Giới hạn

- Cảm biến nhiệt độ nội bộ có độ chính xác $\pm 1.5^{\circ}\text{C}$ theo tài liệu kỹ thuật, phụ thuộc vào hiệu chuẩn nhà máy.
- RTC không có pin backup, reset về thời gian biên dịch mỗi lần nạp firmware.
- Một góc màn hình cảm ứng bị liệt tại góc phải dưới, ảnh hưởng đến thiết kế bố cục UI.

1.4 Yêu cầu sử dụng RTOS trong đồ án

Theo yêu cầu của đề tài, hệ thống phải được triển khai trên nền tảng hệ điều hành thời gian thực (RTOS). Việc áp dụng RTOS nhằm mục tiêu giải quyết các thách thức về quản lý đa nhiệm và đảm bảo tính thời gian thực cho các tác vụ quan trọng. Bảng 1.1 so sánh chi tiết giữa phương pháp RTOS (yêu cầu của đề bài) và phương pháp Super Loop truyền thống để làm rõ ưu thế của kiến trúc được sử dụng.

Bảng 1.1: So sánh Super Loop và RTOS

Tiêu chí	Super Loop	RTOS (FreeRTOS)
Đáp ứng thời gian thực	Phụ thuộc vào tác vụ chạy lâu nhất trong vòng lặp	Đảm bảo bởi mức ưu tiên, task quan trọng luôn được chạy trước
Xử lý đa nhiệm	Giả lập bằng state machine, phức tạp khi mở rộng	Thực sự song song (preemptive scheduling), mỗi task có stack riêng
Bảo vệ tài nguyên chung	Phải làm thủ công bằng flag, dễ xảy ra race condition	Mutex, Semaphore, Thread Flags có sẵn, đã được kiểm chứng
Mở rộng thêm tác vụ	Mỗi tác vụ mới đều ảnh hưởng timing của toàn hệ thống	Thêm task mới đơn giản, không ảnh hưởng task hiện có
Tiêu thụ RAM	Thấp, chỉ sử dụng 1 stack	Mỗi task cần stack riêng từ 256B đến 4KB, cần quản lý heap
Độ phức tạp debug	Đơn giản, tuần tự	Khó hơn, đòi hỏi hiểu biết về lập lịch và quản lý tài nguyên

Việc sử dụng RTOS theo yêu cầu đề bài giúp hệ thống đáp ứng tốt các chức năng: đọc cảm biến liên tục, hiển thị LCD mượt mà, ghi dữ liệu SD Card mà không gây nghẽn hệ thống và xử lý cảm ứng phản hồi nhanh. Chi phí về tài nguyên bộ nhớ khoảng 20 KB cho heap và stack hoàn toàn nằm trong khả năng đáp ứng của MCU STM32F405 (192 KB SRAM).

1.5 Cấu trúc báo cáo

Báo cáo được tổ chức thành 6 chương:

- **Chương 1 – Giới thiệu:** Trình bày đặt vấn đề, mục tiêu, phạm vi đồ án.
- **Chương 2 – Thiết kế phần cứng:** Mô tả chi tiết các thành phần phần cứng, sơ đồ khối, sơ đồ kết nối và cấu hình clock.
- **Chương 3 – Thiết kế phần mềm:** Kiến trúc RTOS, cơ chế đồng bộ hóa, cấu hình ADC, RTC, FatFs.
- **Chương 4 – Triển khai chi tiết:** Code và giải thích từng task: TaskSensor, TaskGUI, TaskSD.
- **Chương 5 – Kết quả thực nghiệm:** Ảnh chụp LCD, dữ liệu SD Card, phân tích hiệu năng.
- **Chương 6 – Kết luận:** Tổng kết kết quả, hạn chế và hướng phát triển.

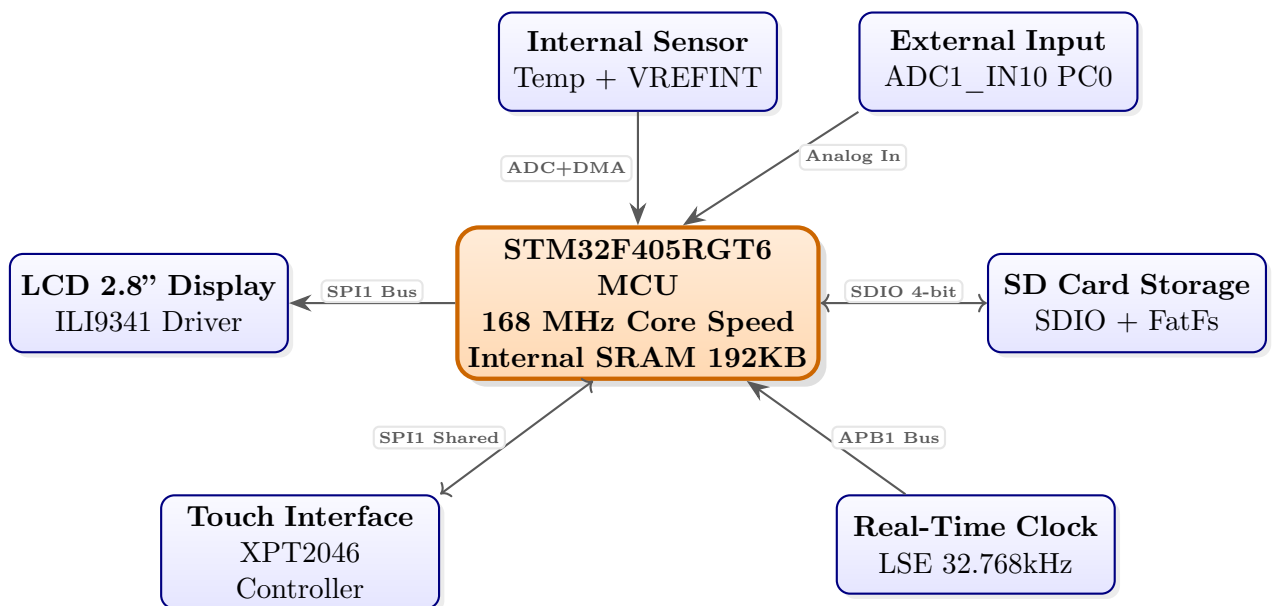
Chương 2

Thiết Kế Phần Cứng

2.1 Tổng quan hệ thống

Hệ thống được xây dựng trên board phát triển **Open405R-C** của Waveshare, tích hợp vi điều khiển STM32F405RGT6 với các ngoại vi chính: cảm biến nhiệt độ nội bộ, màn hình LCD 2.8" cảm ứng điện trở, thẻ nhớ SD Card, RTC, và đầu vào ADC ngoại PC0. Tất cả các thành phần được quản lý bởi FreeRTOS, đảm bảo hoạt động song song không xung đột.

2.2 Sơ đồ khối hệ thống



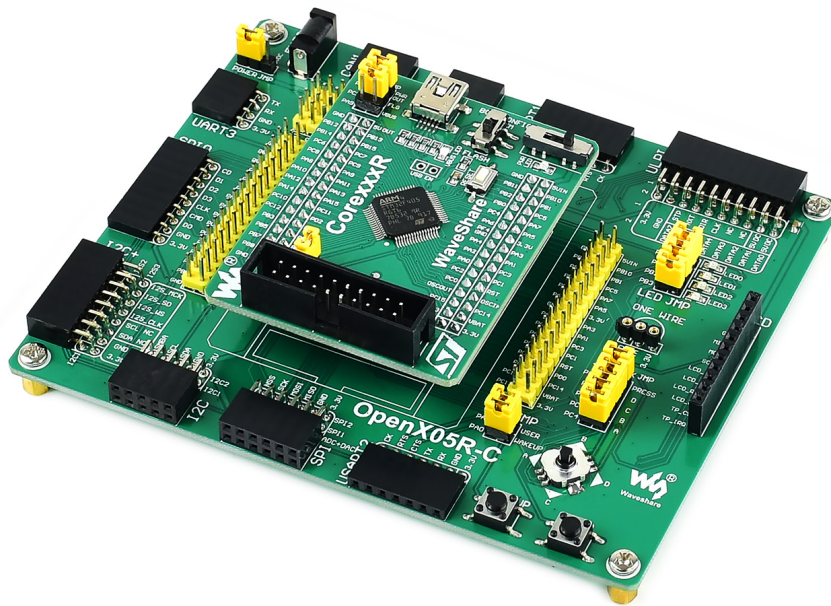
Hình 2.1: Sơ đồ khối hệ thống

2.3 Vi điều khiển STM32F405RGT6

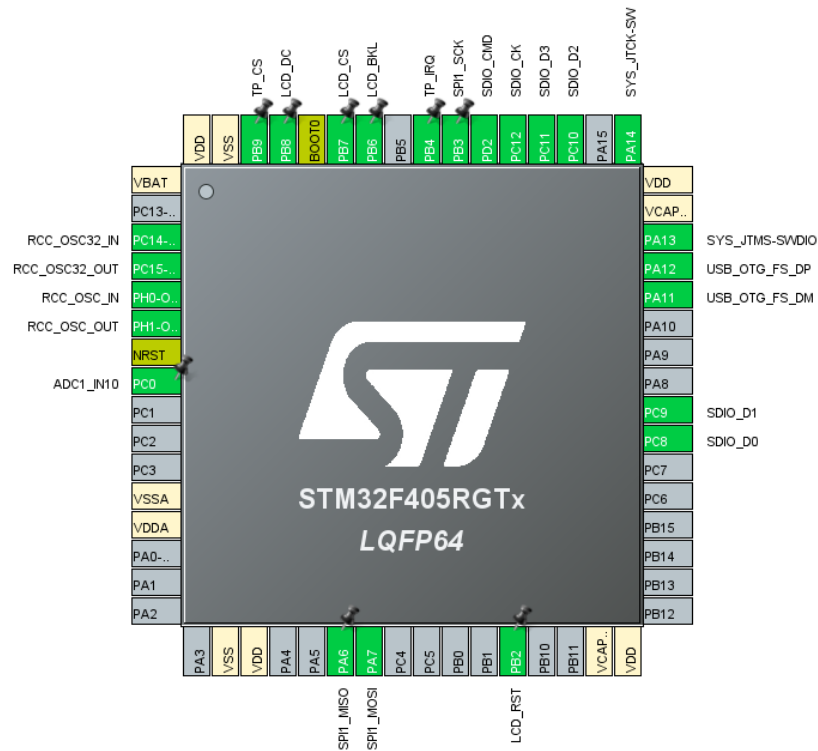
STM32F405RGT6 thuộc dòng STM32F4 hiệu năng cao, dựa trên lõi ARM Cortex-M4 với FPU và DSP instructions. Vi xử lý này hoàn toàn đáp ứng tốt các yêu cầu của đề tài về triển khai ứng dụng đa nhiệm trên nền tảng RTOS nhờ bộ nhớ lớn và tốc độ xử lý cao.

Bảng 2.1: Thông số chính STM32F405RGT6

Thông số	Giá trị
Lõi xử lý	ARM Cortex-M4F, tối đa 168 MHz
Flash (bộ nhớ chương trình)	1 MB
SRAM (bộ nhớ dữ liệu)	192 KB với 128 KB SRAM1/2 và 64 KB CCM
ADC	3 × 12-bit, tối đa 2.4 MSPS
RTC	Lịch thời gian thực, nguồn LSE 32.768 kHz
SDIO	1 × hỗ trợ SD/SDHC, 1-bit và 4-bit
SPI	3 × SPI1 dùng cho LCD và Touch
DMA	2 DMA controller, 16 stream
USB	OTG Full-Speed
Package	LQFP-64
Điện áp hoạt động	1.8V – 3.6V



Hình 2.2: Board Open405R-C thực tế

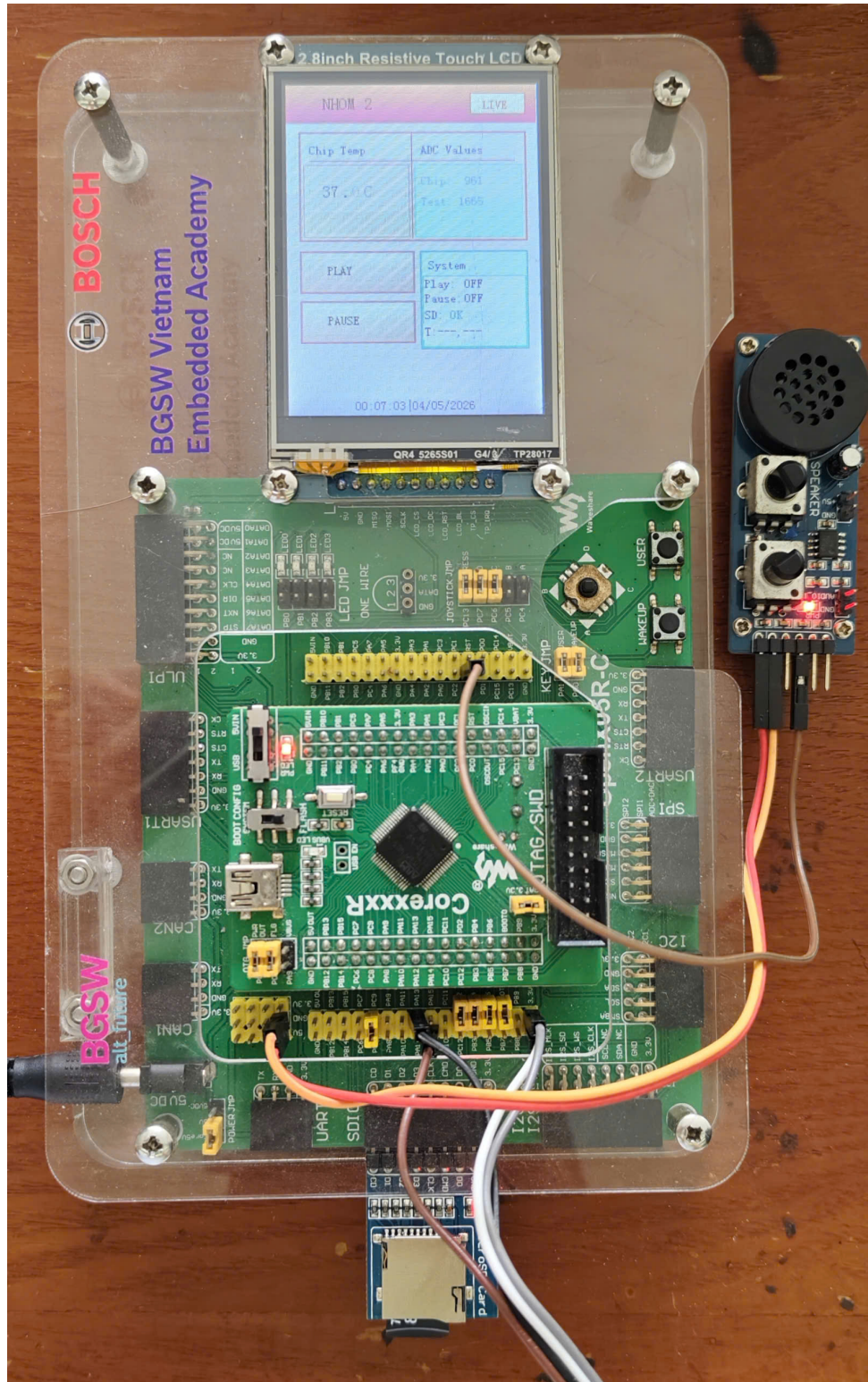


Hình 2.3: Sơ đồ chân STM32F405RGT6

2.4 Board phát triển Open405R-C

Board **Open405R-C** do Waveshare sản xuất, cung cấp sẵn:

- MCU STM32F405RGT6 với thạch anh HSE 25 MHz và LSE 32.768 kHz.
- Khe cắm SD Card kết nối qua SDIO (4-bit).
- Header mở rộng để kết nối LCD, cảm biến, và các module ngoại vi.
- Mạch nạp/debug qua SWD.
- LED chỉ thị nguồn và GPIO.
- Cổng USB OTG Full-Speed (Type-A).



Hình 2.4: Board Open405R-C kết nối với LCD 2.8" và SD Card

2.5 Màn hình LCD 2.8" cảm ứng điện trở

Module LCD 2.8" tích hợp cả màn hình hiển thị và panel cảm ứng điện trở trên cùng một board:

2.5.1 Thông số LCD

Bảng 2.2: Thông số module LCD 2.8"

Thông số	Giá trị
Driver IC	ILI9341
Độ phân giải	240 × 320 pixel
Màu sắc	65K màu (RGB565, 16-bit/pixel)
Giao tiếp	SPI (4-wire)
Tốc độ SPI	Tối đa 42 Mbit/s (Prescaler/2 trên APB2)
Backlight	LED, điều khiển bằng GPIO (PB6)

2.5.2 Thông số Touch Panel

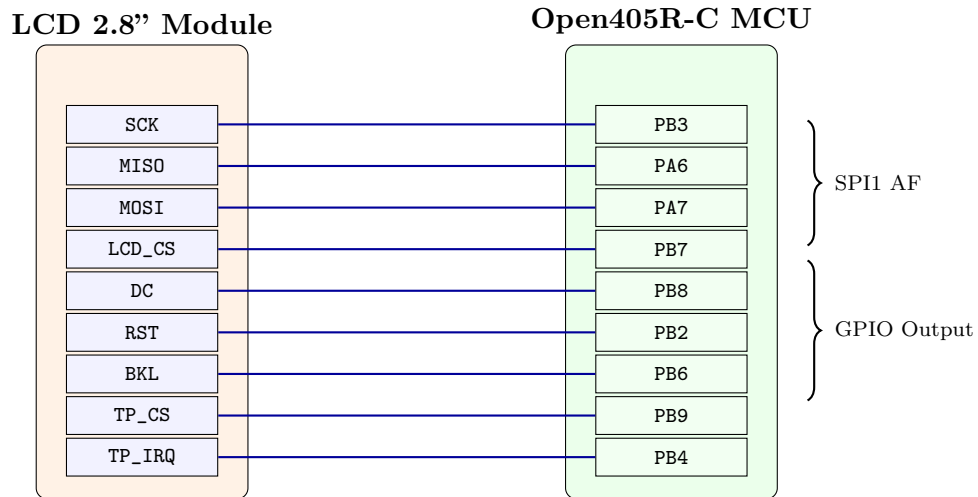
Bảng 2.3: Thông số Touch Panel

Thông số	Giá trị
Controller IC	XPT2046
Loại cảm ứng	Điện trở
Độ phân giải ADC	12-bit (0 – 4095)
Giao tiếp	SPI (chia sẻ bus SPI1 với LCD)
Tốc độ SPI khi đọc	Prescaler/64 (giảm tốc để đảm bảo chính xác)
Chân IRQ	PB4 (Active Low, ngắt khi có chạm)

2.5.3 Sơ đồ kết nối chân

Bảng 2.4: Bảng kết nối chân LCD và Touch

Chức năng	Chân MCU	GPIO Mode	Ghi chú
SPI1_SCK	PB3	Alternate Function	Clock chung LCD + Touch
SPI1_MISO	PA6	Alternate Function	Dữ liệu từ Touch → MCU
SPI1_MOSI	PA7	Alternate Function	Dữ liệu từ MCU → LCD/Touch
LCD_CS	PB7	Output (Active Low)	Chip Select LCD
LCD_DC	PB8	Output	Data/Command select
LCD_RST	PB2	Output	Hardware Reset LCD
LCD_BKL	PB6	Output	Backlight ON/OFF
TP_CS	PB9	Output (Active Low)	Chip Select Touch
TP_IRQ	PB4	Input Pull-Up	Ngắt báo có chạm



Hình 2.5: Sơ đồ kết nối chân LCD Module với board Open405R-C

2.6 Module SD Card

Thẻ nhớ SD Card được kết nối qua giao tiếp **SDIO** với các đặc điểm:

Bảng 2.5: Cấu hình SDIO

Thông số	Giá trị
Bus Width	4-bit (khởi tạo) → 1-bit (thực tế, ổn định hơn)
Clock Divider	10 (khởi tạo), Auto-Tune giảm dần: {10, 8, 6, 4, 2}
DMA RX	DMA2_Stream3
DMA TX	DMA2_Stream6
Data Lines	PC8 (D0), PC9 (D1), PC10 (D2), PC11 (D3)
Clock	PC12 (CK)
Command	PD2 (CMD)
Hệ thống file	FatFs (FAT32)

Tính năng đặc biệt:

- **Auto-Format:** Nếu phát hiện thẻ nhớ chưa có filesystem (FR_NO_FILESYSTEM), hệ thống tự động format FAT32 (tối đa 3 lần thử).
- **Auto-Tune:** Sau khi mount thành công, hệ thống thử tăng dần tốc độ clock SDIO qua các mức divider {10, 8, 6, 4, 2}, kiểm tra filesystem sau mỗi bước. Nếu thất bại, quay về mức ổn định cuối cùng.
- **Ưu tiên 1-bit mode:** Trên board Open405R-C, tín hiệu SDIO 4-bit có thể bị nhiễu do PCB layout. Hệ thống ưu tiên khởi tạo ở chế độ 1-bit để tăng độ ổn định.

2.7 ADC ngoại

Ngoài cảm biến nhiệt độ nội, hệ thống đọc thêm kênh **ADC1_IN10** trên chân **PC0**. Kênh này có thể kết nối với:

- Biến trở để test nhanh.
- Cảm biến analog bên ngoài (LM35, LM335, v.v.).
- Mạch chia áp để đo điện áp.

Sampling time của kênh IN10 là 84 cycles (ngắn hơn kênh nhiệt độ 112 cycles vì không yêu cầu thời gian charge capacitor dài).

2.8 Cấu hình Clock

Hệ thống clock là “trái tim” của toàn bộ hoạt động vi điều khiển. Cấu hình sai sẽ dẫn đến tất cả ngoại vi hoạt động sai.

2.8.1 PLL Configuration

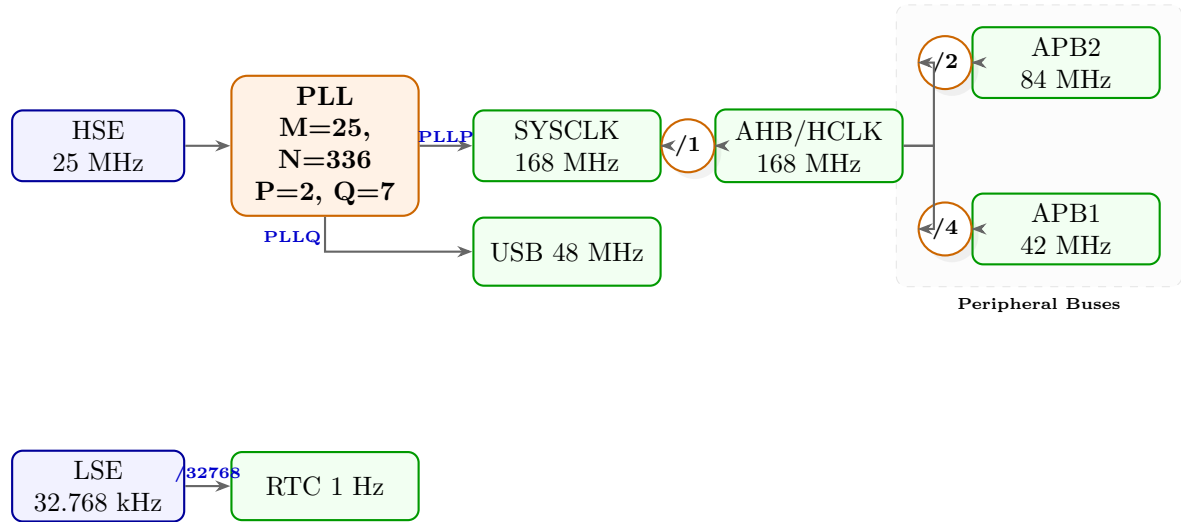
Clock hệ thống được cấu hình qua PLL từ thạch anh ngoại HSE 25 MHz:

$$f_{SYSCLK} = \frac{HSE \times PLLN}{PLLM \times PLLP} = \frac{25 \times 336}{25 \times 2} = 168 \text{ MHz} \quad (2.1)$$

2.8.2 Clock Tree

Bảng 2.6: Bảng tần số các bus

Bus	Bộ chia	Tần số	Ngoại vi sử dụng
SYSCLK	PLL	168 MHz	CPU core
HCLK (AHB)	/1	168 MHz	DMA, Flash, GPIO
APB1	/4	42 MHz	RTC, SDIO, Timer
APB2	/2	84 MHz	SPI1 (LCD + Touch), ADC1
USB (48 MHz)	PLLQ=7	48 MHz	USB OTG Full-Speed
RTC	LSE	32.768 kHz	Real-Time Clock
ADC	APB2/4	21 MHz	ADC sampling clock



Hình 2.6: Sơ đồ Clock Tree hệ thống

2.8.3 Cấu hình nguồn clock trong code

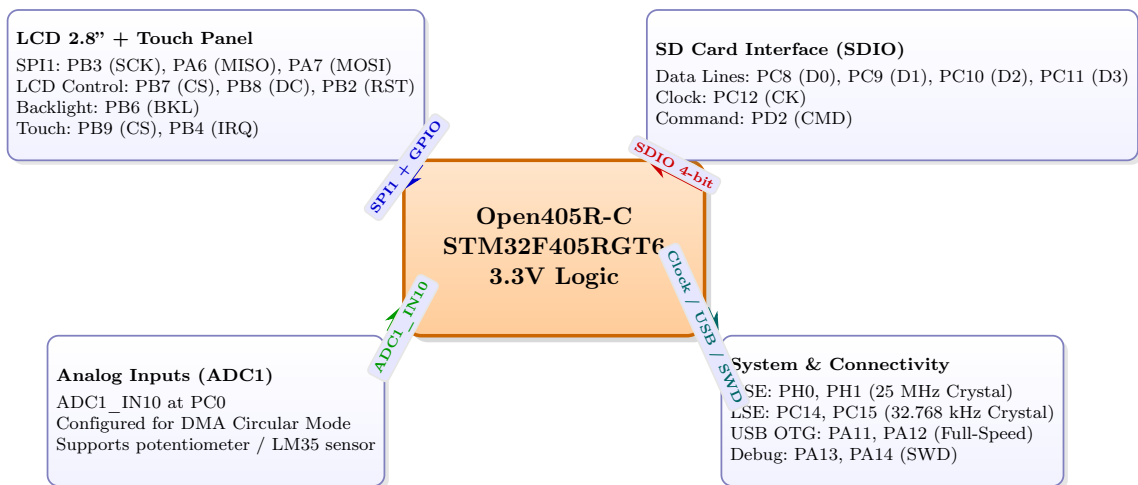
```

1 RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE
2                               | RCC_OSCILLATORTYPE_LSE;
3 RCC_OscInitStruct.HSEState = RCC_HSE_ON;
4 RCC_OscInitStruct.LSEState = RCC_LSE_ON;
5 RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
6 RCC_OscInitStruct.PLL.PLLM = 25; // VCO input = 1 MHz
7 RCC_OscInitStruct.PLL.PLLN = 336; // VCO output = 336 MHz
8 RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2; // SYSCLK = 168 MHz
9 RCC_OscInitStruct.PLL.PLLQ = 7; // USB = 48 MHz

```

Listing 2.1: SystemClock_Config — Cấu hình PLL và bus clock

2.9 Sơ đồ kết nối tổng thể



Hình 2.7: Sơ đồ nối dây tổng thể của hệ thống theo cấu hình `doan.ioc`

Sơ đồ trong Hình 2.7 được tổng hợp trực tiếp từ file cấu hình `doan.ioc` và mã nguồn HAL của đồ án. Điểm cần lưu ý là bus **SPI1** dùng chung cho LCD và Touch, trong đó chân xung clock thực tế là **PB3**; ngõ vào analog ngoài sử dụng **PC0 / ADC1_IN10**; còn thẻ nhớ SD nối qua **SDIO** với các chân **PC8–PC12** và **PD2**.

Bảng 2.7: Tổng hợp chân GPIO sử dụng trong đồ án

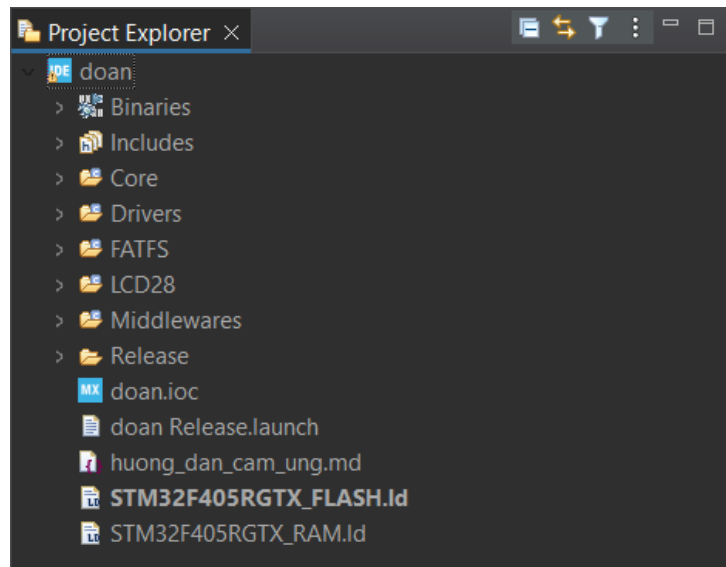
Chân	Ngoại vi	Chức năng	Mode
PA6	SPI1_MISO	Dữ liệu từ Touch	AF
PA7	SPI1_MOSI	Dữ liệu đến LCD/Touch	AF
PA11/PA12	USB_OTG_FS	USB Device	AF
PA13/PA14	SYS	SWD Debug	AF
PB2	LCD_RST	Reset LCD	Output
PB3	SPI1_SCK	Clock SPI	AF
PB4	TP_IRQ	Ngắt cảm ứng	EXTI, Pull-Up
PB6	LCD_BKL	Backlight LCD	Output
PB7	LCD_CS	Chip Select LCD	Output
PB8	LCD_DC	Data/Command LCD	Output
PB9	TP_CS	Chip Select Touch	Output
PC0	ADC1_IN10	ADC ngoại (analog)	Analog
PC8-PC11	SDIO_D0-D3	Data bus SD	AF
PC12	SDIO_CK	Clock SD	AF
PC14/PC15	RCC_OSC32	LSE Crystal	OSC
PD2	SDIO_CMD	Command SD	AF
PH0/PH1	RCC_OSC	HSE Crystal	OSC

Chương 3

Thiết Kế Phần Mềm

3.1 Tổng quan kiến trúc phần mềm

Tuân thủ yêu cầu của đề tài, phần mềm hệ thống được thiết kế theo kiến trúc **multi-task RTOS**, trong đó mỗi chức năng chính được giao cho một task FreeRTOS độc lập. Các task giao tiếp với nhau thông qua các cơ chế đồng bộ hóa chuẩn của CMSIS-RTOS v2: Mutex, Thread Notification Flags, và biến volatile chia sẻ.



Hình 3.1: Cấu trúc project trong STM32CubeIDE

3.1.1 Cấu trúc thư mục project

```
doan/
|-- Core/
|   |-- Inc/           -- Header files (main.h, FreeRTOSConfig.h)
|   '-- Src/          -- Source files (main.c, freertos.c)
|-- LCD28/            -- Driver LCD ILI9341 + Touch XPT2046
|   |-- LCD_Driver.c  -- Ham dieu khien LCD (SPI)
|   |-- LCD_lib.c     -- Font, ve hinh, text
|   '-- TIM3_PWM.c    -- Backlight PWM (neu dung)
|-- FATFS/            -- Middleware FatFs
```

```
|  |-- App/          -- fatfs.c (init + link driver)
|  '-- Target/       -- bsp_driver_sd.c (HAL SD adapter)
|-- Drivers/         -- STM32 HAL Library
|-- Middlewares/     -- FreeRTOS kernel
'-- doan.ioc         -- Cau hình CubeMX
```

3.2 Kiến trúc FreeRTOS

Hệ thống sử dụng **FreeRTOS** qua CMSIS-RTOS v2 API, bộ lập lịch preemptive với 3 task chính. Bảng 3.1 mô tả chi tiết từng task.

Bảng 3.1: Danh sách Task trong hệ thống

Task	Hàm	Priority	Stack	Chức năng
TaskSensor	StartTaskSensor	High (40)	1 KB	Đọc 3 kênh ADC (Temp, VRE-FINT, IN10), lọc EMA, tính nhiệt độ
TaskSD	StartTaskSD	AboveNormal (32)	4 KB	Khởi tạo SD Card, ghi/đọc file FatFs, auto-format, auto-tune
TaskGUI	StartTaskGUI	Normal (24)	4 KB	Vẽ LCD, xử lý cảm ứng, xem dữ liệu, quản lý trạng thái UI

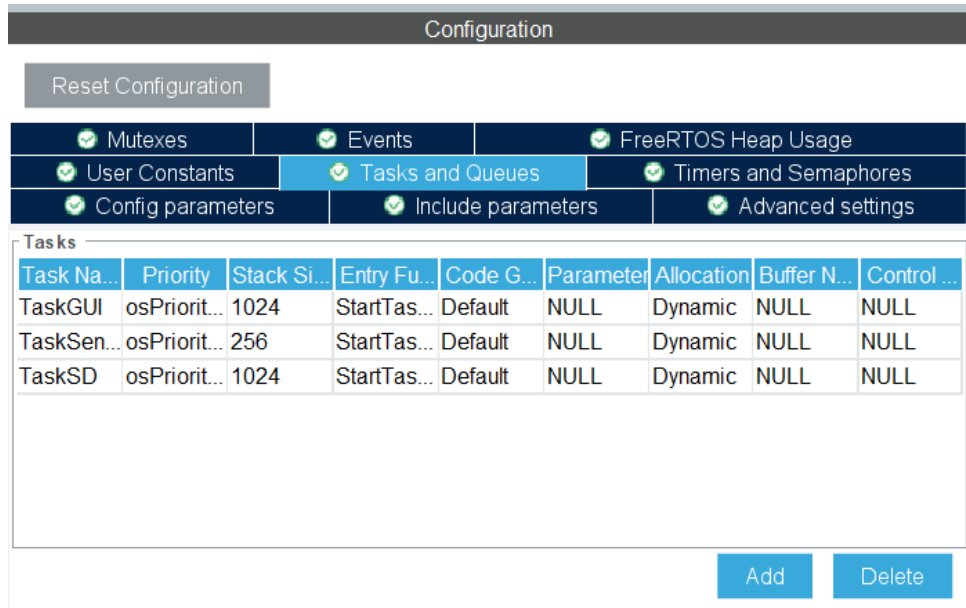
Nguyên tắc phân ưu tiên:

- **TaskSensor (High):** Ưu tiên cao nhất vì dữ liệu cảm biến phải luôn được cập nhật, không bị miss mẫu.
- **TaskSD (AboveNormal):** Cao hơn GUI vì thao tác file I/O cần hoàn thành nhanh để tránh mất dữ liệu.
- **TaskGUI (Normal):** Thấp nhất vì giao diện có thể chậm vài ms mà người dùng không nhận ra.

3.2.1 Cấu hình FreeRTOS Heap

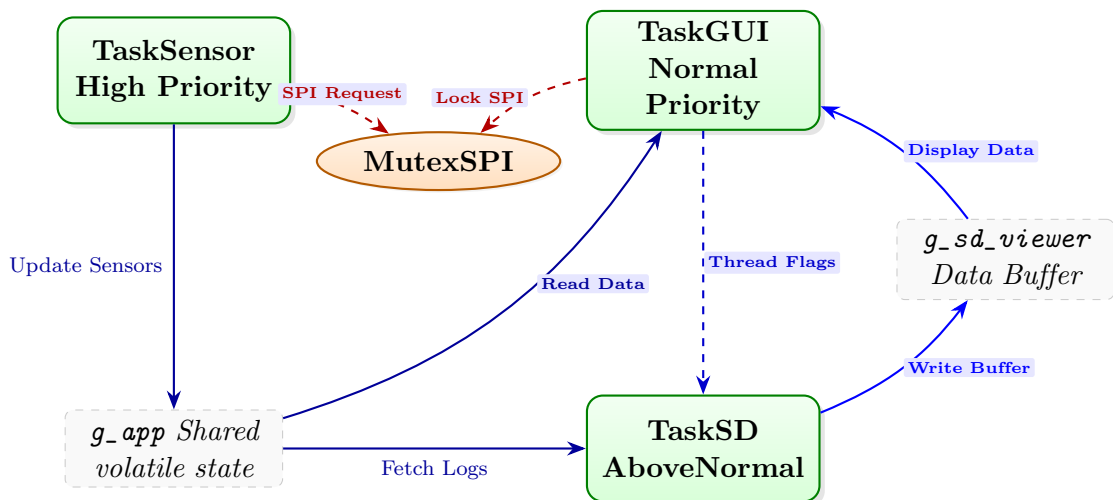
Bảng 3.2: Cấu hình bộ nhớ FreeRTOS

Thông số	Giá trị
Total Heap Size	32 KB (configTOTAL_HEAP_SIZE = 32768)
Heap scheme	heap_4 (best fit + merge)
Newlib Reentrant	Enabled (configUSE_NEWLIB_REENTRANT = 1)
HAL Timebase	TIM1 (không dùng SysTick cho HAL)



Hình 3.2: Cấu hình FreeRTOS Tasks trong CubeMX

3.3 Sơ đồ tương tác giữa các Task



Hình 3.3: Sơ đồ tương tác giữa các Task và tài nguyên hệ thống

3.4 Cơ chế đồng bộ hóa

Hệ thống sử dụng 3 cơ chế đồng bộ hóa khác nhau, mỗi cơ chế phù hợp với một mục đích cụ thể:

3.4.1 Mutex SPI (MutexSPI)

LCD và Touch Panel chia sẻ bus SPI1. Nếu không có mutex, TaskGUI có thể đang gửi lệnh vẽ LCD (CS_LCD = LOW) thì bị preempt, dẫn đến touch đọc sai. MutexSPIHandle đảm bảo tại mọi thời điểm chỉ có 1 task truy cập SPI1.

```

1 if (osMutexAcquire(MutexSPIHandle, 50) == osOK) {
2     // Toàn bộ thao tác LCD và Touch an toàn trong mutex
3     ui_handle_touch_and_buttons();
4     ui_refresh_dynamic();
5     osMutexRelease(MutexSPIHandle);
6 }
7 osDelay(2); // Nhường CPU cho task khác

```

Listing 3.1: TaskGUI sử dụng Mutex SPI

3.4.2 Thread Notification Flags

TaskGUI điều khiển TaskSD bằng `osThreadFlagsSet/Wait` — cơ chế zero-copy nhẹ hơn Message Queue, phù hợp khi chỉ cần gửi “tín hiệu điều khiển” mà không cần truyền dữ liệu.

```

1 #define SD_FLAG_PLAY_START    0x01u // Bat dau ghi
2 #define SD_FLAG_PLAY_STOP    0x02u // Dung ghi
3 #define SD_FLAG_READ_REQUEST 0x04u // Doc file hien thi
4 #define SD_FLAG_DELETE       0x08u // Xoa file data.txt
5 #define SD_FLAG_ALL           (SD_FLAG_PLAY_START | SD_FLAG_PLAY_STOP \
6                               | SD_FLAG_READ_REQUEST | SD_FLAG_DELETE)

```

Listing 3.2: Định nghĩa các Flag điều khiển

Vấn đề Phantom Trigger

Hiện tượng “Phantom Trigger” nảy sinh từ sự trùng lặp giữa mã lỗi hệ thống và các định nghĩa flag của ứng dụng. Cụ thể, khi hàm `osThreadFlagsWait` hết thời gian chờ (timeout), giá trị trả về mang định dạng mã lỗi `0xFFFFFFFFx`. Sự giao thoa giữa các bit trọng số thấp của mã lỗi này và các giá trị `SD_FLAG_*` khiến hệ thống hiểu nhầm mã lỗi là một thông báo flag hợp lệ. Vấn đề này đã được phân tích và xử lý triệt để trong giai đoạn triển khai phần mềm.

```

1 /* Ma loi CMSIS-RTOS luôn có bit MSB = 1.
2    Flags hợp lệ chỉ sử dụng bit 0-3, nên bit MSB luôn = 0.
3    => Kiểm tra bit 31 để phân biệt. */
4 #define SD_FLAGS_VALID(f) (((f) & 0x80000000u) == 0u)
5
6 flags = osThreadFlagsWait(SD_FLAG_ALL, osFlagsWaitAny, 100);
7 if (SD_FLAGS_VALID(flags)) {
8     // Chỉ xử lý khi flags thật sự hợp lệ
9     if (flags & SD_FLAG_PLAY_START) { /* ... */ }
10 }

```

Listing 3.3: Bảo vệ khỏi Phantom Trigger

3.4.3 Biến chia sẻ volatile

Cấu trúc `g_app` (kiểu `app_state_t`) được khai báo `volatile` để compiler không tối ưu hóa bỏ qua các lần đọc. Với kiến trúc ARM Cortex-M4, các thao tác đọc/ghi 32-bit trên vùng nhớ aligned đã là **atomic**, nên không cần thêm mutex cho biến này.

```

1 typedef struct {
2     uint16_t adc_raw_temp;      // ADC tho nhiet do chip
3     uint16_t adc_raw_analog;    // ADC tho kenh ngoai (IN10)
4     int16_t temp_x10;           // Nhiet do x10 (VD: 253 = 25.3 C)
5     bool play_enabled;         // Dang ghi file?
6     bool pause_enabled;        // Dang xem Data Viewer?
7     bool touch_pressed;        // Co dang cham man hinh?
8     uint16_t touch_x, touch_y; // Toa do cham (pixel)
9     sd_state_t sd_state;        // Trang thai SD Card
10    uint16_t sd_code;           // Ma loi chi tiet
11 } app_state_t;
12
13 static volatile app_state_t g_app = { /* ... */ };

```

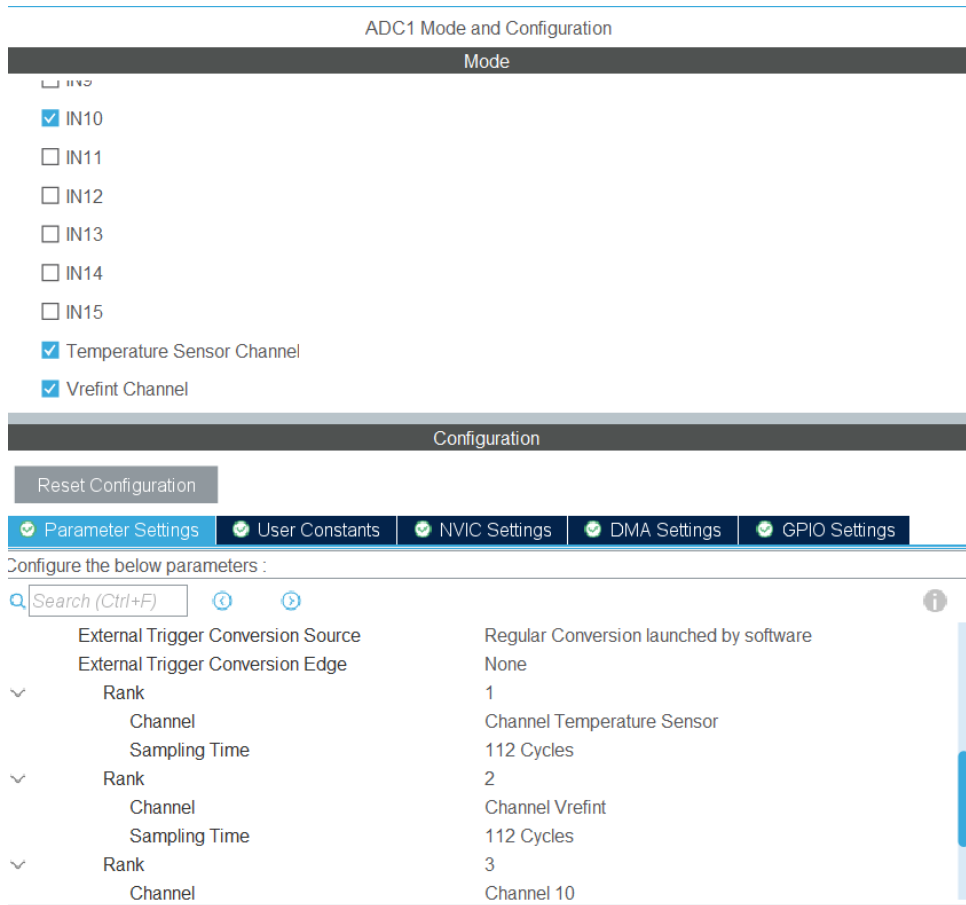
Listing 3.4: Cấu trúc app_state_t — Trạng thái chia sẻ giữa các task

3.5 Cấu hình ADC

ADC1 được cấu hình đọc **3 kênh** liên tục bằng DMA (Circular mode), tự động chuyển đổi không cần CPU can thiệp:

Bảng 3.3: Cấu hình kênh ADC

Rank	Kênh	Sampling Time	Chức năng
1	TEMPSENSOR	112 cycles	Đọc nhiệt độ nhân chip (junction temp)
2	VREFINT	112 cycles	Điện áp tham chiếu nội để bù sụt áp nguồn
3	IN10 (PC0)	84 cycles	ADC ngoại (biến trở / cảm biến bên ngoài)



Hình 3.4: Cấu hình ADC trong CubeMX

```

1  /* Buffer DMA: [0]=Temp, [1]=VREFINT, [2]=IN10 */
2  static volatile uint16_t g_adc_dma[3] = {0u, 0u, 0u};
3
4  /* Bat dau chuyen doi ADC lien tuc bang DMA Circular */
5  HAL_ADC_Start_DMA(&hadc1, (uint32_t *)g_adc_dma, 3u);

```

Listing 3.5: Khởi động ADC DMA cho 3 kênh

3.5.1 Công thức tính nhiệt độ

Nhiệt độ junction (nhiệt độ nhân chip) được tính bằng phương pháp **nội suy tuyến tính 2 điểm hiệu chuẩn nhà máy** (30°C và 110°C), kết hợp bù trừ sụt áp nguồn (VREFINT):

Bước 1: Bù trừ sụt áp nguồn:

$$ADC_{corrected} = ADC_{raw} \times \frac{VREFINT_CAL}{VREFINT_DATA} \quad (3.1)$$

Trong đó:

- VREFINT_CAL = giá trị hiệu chuẩn VREFINT tại 3.3V (lưu ở 0x1FFF7A2A).
- VREFINT_DATA = giá trị VREFINT đo thực tế (kênh DMA rank 2).

Bước 2: Nội suy nhiệt độ:

$$T_{junction} = 30 + \frac{80 \times (ADC_{corrected} - TS_CAL1)}{TS_CAL2 - TS_CAL1} \quad [^{\circ}C] \quad (3.2)$$

Trong đó:

- TS_CAL1 = giá trị ADC hiệu chuẩn tại 30°C (0x1FFF7A2C).
- TS_CAL2 = giá trị ADC hiệu chuẩn tại 110°C (0x1FFF7A2E).

```

1 static int16_t adc_temp_x10_from_raw(uint16_t raw_temp,
2                                     uint16_t raw_vref) {
3     int32_t cal1 = (int32_t)ADC_TS_CAL1; // 30 do C
4     int32_t cal2 = (int32_t)ADC_TS_CAL2; // 110 do C
5     uint16_t vref_cal = ADC_VREF_CAL;
6
7     if (cal2 <= cal1 || raw_vref == 0) return 250;
8
9     // Bu tru sut ap nguon
10    int32_t raw_temp_corr = (int32_t)raw_temp
11                            * vref_cal / raw_vref;
12
13    // Noi suy nhiet do (x10 de giu 1 chu so thap phan)
14    // VD: 253 = 25.3 do C
15    int32_t temp_x10 = 300
16        + ((800 * (raw_temp_corr - cal1)) / (cal2 - cal1));
17
18    return (int16_t)temp_x10;
19 }

```

Listing 3.6: Hàm tính nhiệt độ với bù VREFINT — Kết quả x10

3.6 Cấu hình RTC

RTC (Real-Time Clock) sử dụng thạch anh ngoại LSE 32.768 kHz, định dạng 24h. Prescaler được cấu hình theo chuẩn:

- Async Predivider: 127
- Sync Predivider: 255
- $\Rightarrow f_{RTC} = \frac{32768}{(127+1) \times (255+1)} = 1 \text{ Hz}$

3.6.1 Tự động đồng bộ thời gian từ thời gian biên dịch

Vì không có pin backup cho RTC, thời gian sẽ bị reset mỗi lần mất nguồn. Để giảm bất tiện, hệ thống tự động cập nhật RTC từ macro biên dịch `__TIME__` và `__DATE__` mỗi khi nạp firmware:

```

1 /* __TIME__ co dinh dang "HH:MM:SS" */
2 const char *t = __TIME__;
3 sTime.Hours   = (t[0] - '0') * 10 + (t[1] - '0');
4 sTime.Minutes = (t[3] - '0') * 10 + (t[4] - '0');

```

```

5 sTime.Seconds = (t[6]-'0')*10 + (t[7]-'0');
6 HAL_RTC_SetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
7
8 /* __DATE__ có định dạng "Mmm dd yyyy" (VD: "Apr 26 2026") */
9 const char *d = __DATE__;
10 uint8_t month = 1;
11 if (d[0] == 'J') {
12     if (d[1] == 'a') month = 1;      // Jan
13     else if (d[2] == 'n') month = 6; // Jun
14     else month = 7;                  // Jul
15 } else if (d[0] == 'F') month = 2;    // Feb
16 /* ... (xu lý các tháng còn lại) ... */
17 HAL_RTC_SetDate(&hrtc, &sDate, RTC_FORMAT_BIN);

```

Listing 3.7: Tự động đồng bộ RTC từ thời gian biên dịch

3.7 Cấu hình FatFs

Bảng 3.4: Cấu hình FatFs

Thông số	Giá trị
Filesystem type	FAT32
Long File Name (LFN)	Enabled (heap-based, _USE_LFN=3)
Max Sector Size	512 bytes
SD Interface	SDIO

3.8 Driver LCD và Touch Panel

3.8.1 Thư viện LCD28

Thư viện LCD28 được điều chỉnh từ bộ demo chính chủ của Waveshare (Open405R) với các thay đổi quan trọng:

Bảng 3.5: So sánh driver gốc và driver đã điều chỉnh

Đặc điểm	Driver gốc (Waveshare)	Driver đồ án (Đã điều chỉnh)
Thư viện HAL	Standard Peripheral Library	STM32 HAL Library
Quản lý bus SPI	Đơn nhiệm cho LCD	Chia sẻ SPI1 giữa LCD và Touch qua CS riêng
Tốc độ SPI	Cố định	Chuyển đổi linh hoạt (Prescaler/2 cho LCD, Prescaler/64 cho Touch)
Xử lý nhiễu touch	Cơ bản hoặc không có	Lọc trung vị + kiểm tra độ ổn định (Stability Span)
Tích hợp RTOS	Không	Tích hợp sâu vào FreeRTOS Task, mutex SPI

3.8.2 Các hàm LCD chính

- `lcd_init()`: Khởi tạo ILI9341 qua SPI, cấu hình orientation, bật backlight.
- `lcd_clear_screen(color)`: Xóa toàn bộ màn hình với màu nền.
- `lcd_fill_rect(x,y,w,h,color)`: Tô hình chữ nhật đặc.
- `lcd_draw_rect(x,y,w,h,color)`: Vẽ khung hình chữ nhật.
- `lcd_draw_h_line(x,y,w,color)`: Vẽ đường ngang.
- `lcd_display_string(x,y,str,font,color)`: Hiển thị chuỗi text.
- `lcd_display_num(x,y,num,len,font,color)`: Hiển thị số nguyên.

3.9 Cấu trúc dữ liệu xem dữ liệu

Chế độ xem dữ liệu sử dụng cấu trúc `sd_viewer_t` theo mô hình **single-writer / single-reader**: chỉ TaskSD ghi vào buffer, chỉ TaskGUI đọc. Cờ `volatile bool ready` đủ để đồng bộ mà không cần mutex riêng.

```

1 #define SD_VIEWER_LINES_PER_PAGE 6u
2 #define SD_VIEWER_LINE_MAX_LEN 56u
3
4 typedef struct {
5     char      lines[6][56]; // 6 dòng, mỗi dòng 56 ký tự
6     uint16_t  line_count;   // Số dòng hợp lệ trang hiện tại
7     uint16_t  total_lines;  // Tổng số dòng trong file
8     uint16_t  current_page; // Trang hiện tại (0-based)
9     uint16_t  total_pages;  // Tổng số trang
10    volatile bool ready;     // TaskSD đã đọc xong
11    volatile bool request_page; // GUI yêu cầu chuyển trang
12    uint16_t  requested_page; // Số trang GUI muốn xem
13 } sd_viewer_t;

```

Listing 3.8: Cấu trúc `sd_viewer_t`

Chương 4

Triển Khai Chi Tiết

4.1 TaskSensor

TaskSensor có mức ưu tiên **High**, chu kỳ 20ms, thực hiện:

1. Đọc 3 giá trị ADC từ buffer DMA: g_adc_dma[0] (Temp), g_adc_dma[1] (VREFINT), g_adc_dma[2] (IN10).
2. Áp dụng bộ lọc **EMA** cho cả 3 kênh để giảm nhiễu:

$$y[n] = \frac{7 \times y[n-1] + 3 \times x[n]}{10} \quad (4.1)$$

Hệ số EMA 7/3 tương đương time constant khoảng 3 mẫu, đủ để san phẳng nhiễu spike mà không gây trễ đáng kể.

```
1 void StartTaskSensor(void *argument) {
2     uint16_t raw_int, raw_vref, raw_analog;
3     uint16_t filt_int = 0, filt_vref = 0, filt_analog = 0;
4     bool filter_primed = false;
5
6     for(;;) {
7         raw_int = g_adc_dma[0];
8         raw_vref = g_adc_dma[1];
9         raw_analog = g_adc_dma[2];
10
11         if (!filter_primed) {
12             filt_int = raw_int;
13             filt_vref = raw_vref;
14             filt_analog = raw_analog;
15             filter_primed = true;
16         } else {
17             // Bộ lọc EMA cho cả 3 kênh
18             filt_int = (filt_int*7 + raw_int*3) / 10;
19             filt_vref = (filt_vref*7 + raw_vref*3) / 10;
20             filt_analog = (filt_analog*7 + raw_analog*3) / 10;
21         }
22
23         g_app.adc_raw_temp = filt_int;
24         g_app.adc_raw_analog = filt_analog;
25         g_app.temp_x10 = adc_temp_x10_from_raw(
```

```

26         filt_int, filt_vref);
27     osDelay(20);
28 }
29 }

```

Listing 4.1: TaskSensor — Vòng lặp chính (3 kênh ADC)

4.2 TaskGUI — Giao diện và xử lý cảm ứng

4.2.1 Thiết kế giao diện

Giao diện được thiết kế với phong cách **Modern Card UI**, sử dụng gradient, bo góc và bảng màu ấm. Tất cả màu được định nghĩa bằng macro RGB565:

```

1 #define RGB565(r,g,b) (((uint16_t)((((r)&0xF8)<<8) \
2 | (((g)&0xFC)<<3) | ((b)>>3))))
3 #define UI_BG          RGB565(255, 244, 235) // Nen am
4 #define UI_ACCENT      RGB565(155, 96, 244)  // Tim nhan
5 #define UI_OK          RGB565(63, 178, 106)  // Xanh la
6 #define UI_ERR         RGB565(230, 82, 82)   // Do loi
7 #define UI_WARN        RGB565(255, 138, 76)  // Cam canh bao

```

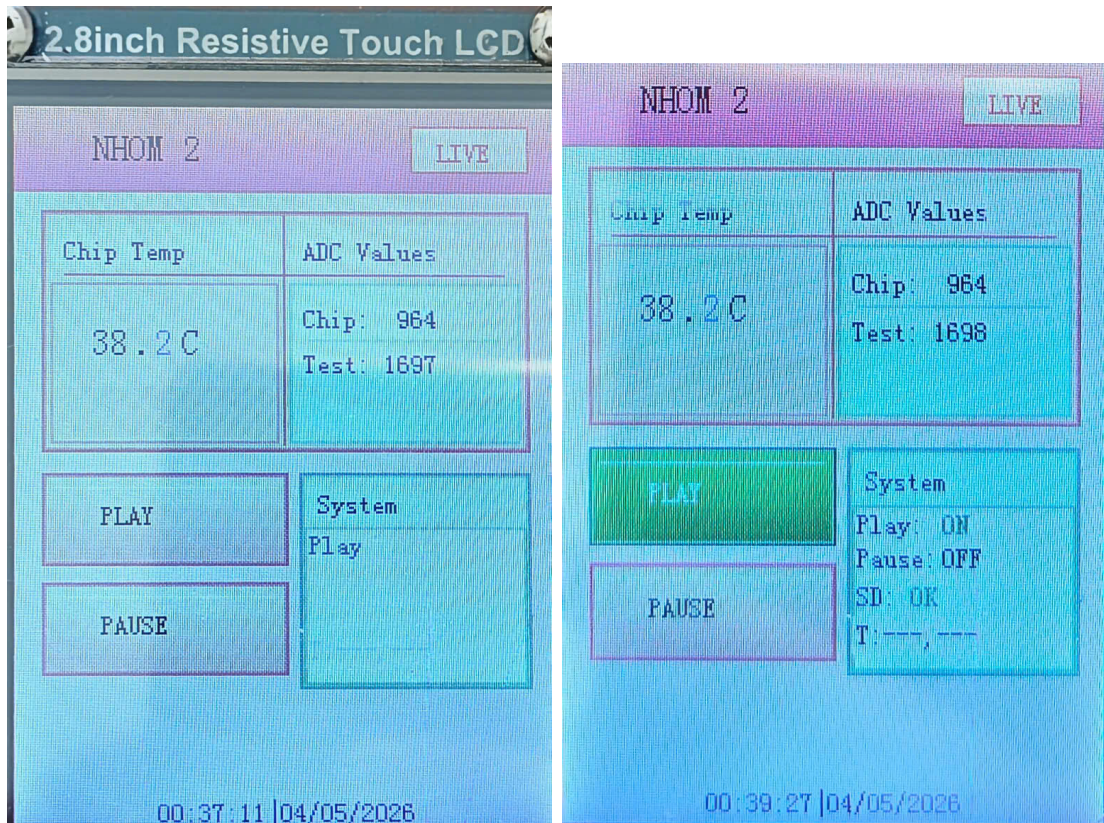
Listing 4.2: Một số macro màu giao diện

4.2.2 Giao diện chính

Màn hình chính được chia thành các vùng chức năng:

Bảng 4.1: Bố cục giao diện LCD chính

Vùng	Tọa độ (Y)	Nội dung
Header	0–37	Gradient cam-hồng, tên “NHOM 2”, badge “LIVE”
Card đo lường	46–152	2 cột: Chip Temp (XX.X°C) và ADC Values (Chip + Test)
Nút PLAY	162–202	Gradient xanh lá khi active, trắng kem khi idle
Nút PAUSE	210–250	Gradient cam khi active, trắng kem khi idle
System Info	162–256 (phải)	Play ON/OFF, Pause ON/OFF, SD status, tọa độ touch
Footer	305–320	Đồng hồ RTC: HH:MM:SS DD/MM/YYYY



Hình 4.1: Giao diện Home (trái) và khi đang PLAY (phải)

4.2.3 Kỹ thuật vẽ Gradient trên LCD

Hệ thống sử dụng hàm nội suy màu RGB565 để tạo hiệu ứng gradient mượt mà trên các nút và vùng nền:

```

1 static uint16_t ui_lerp565(uint16_t c0, uint16_t c1,
2                             uint16_t step, uint16_t max_step) {
3     uint16_t r0 = (c0 >> 11) & 0x1F;
4     uint16_t g0 = (c0 >> 5) & 0x3F;
5     uint16_t b0 = c0 & 0x1F;
6     // Nội suy tuyến tính từng kênh màu
7     uint16_t r = (r0*(max_step-step) + r1*step) / max_step;
8     // ... tương tự cho g, b
9     return (r << 11) | (g << 5) | b;
10 }
11
12 static void ui_fill_gradient_rect(uint16_t x, uint16_t y,
13                                   uint16_t w, uint16_t h, uint16_t top, uint16_t bottom) {
14     for (uint16_t line = 0; line < h; line++) {
15         uint16_t c = ui_lerp565(top, bottom, line, h-1);
16         lcd_draw_h_line(x, y+line, w, c);
17     }
18 }

```

Listing 4.3: Hàm nội suy màu RGB565 và vẽ gradient

4.2.4 Xử lý cảm ứng

Touch panel XPT2046 được đọc qua SPI với quy trình lọc nhiễu 2 lớp:

Lớp 1 — Lọc trung vị: Đọc 3 mẫu liên tiếp cho mỗi trục, lấy giá trị giữa (median) để loại bỏ spike đột ngột.

```

1 static uint16_t median3_u16(uint16_t a, uint16_t b, uint16_t c){
2     if (a > b) { uint16_t t=a; a=b; b=t; }
3     if (b > c) { uint16_t t=b; b=c; c=t; }
4     if (a > b) { uint16_t t=a; a=b; b=t; }
5     return b; // Giá trị ở giữa
6 }

```

Listing 4.4: Lọc trung vị 3 điểm

Lớp 2 — Kiểm tra độ ổn định: Chỉ chấp nhận tọa độ nếu $X_{max} - X_{min} \leq 180$ với ngưỡng TOUCH_STABLE_SPAN_MAX. Nếu vượt quá, bỏ qua lần chạm.

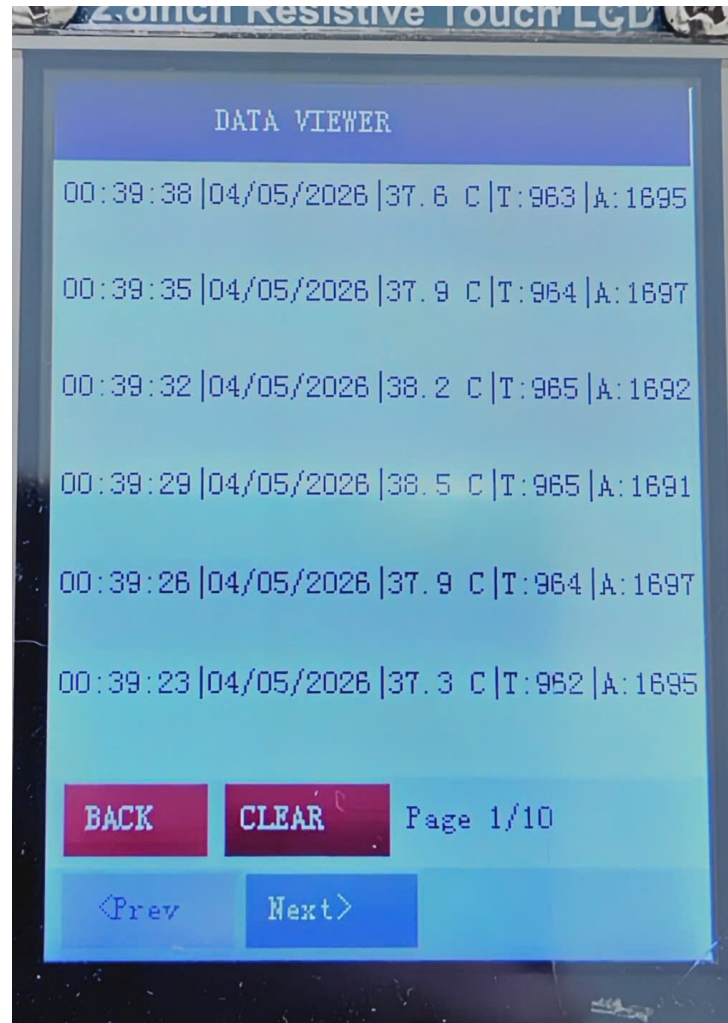
Ánh xạ tọa độ:

$$X_{screen} = \frac{(X_{raw} - X_{min}) \times 239}{X_{max} - X_{min}} \quad (4.2)$$

Cơ chế press-and-release: Chỉ kích hoạt hành động khi người dùng *thả tay* tại đúng nút đã nhấn ban đầu, tránh kích hoạt nhầm khi kéo ngón tay qua nhiều nút.

4.2.5 Chế độ Data Viewer

Khi nhấn PAUSE, TaskGUI chuyển sang chế độ xem dữ liệu với giao diện riêng:



Hình 4.2: Giao diện Data Viewer trên LCD

- Gửi `SD_FLAG_READ_REQUEST` cho TaskSD đọc file.
- Hiển thị dữ liệu theo trang gồm 6 dòng mỗi trang với nền xen kẽ.
- Dữ liệu **mới nhất lên đầu** bằng cách đảo ngược thứ tự đọc file.
- Nút BACK màu đỏ: quay về Home. Nút CLEAR: xóa file data.txt.
- Nút Trước/Sau: chuyển trang.
- **Safe Zone:** Tất cả nút điều hướng đặt ở **nửa trái** màn hình nhằm tránh vùng cảm ứng bị liệt ở góc phải dưới.

4.2.6 Cơ chế Smart Redraw

Để tránh vẽ lại LCD không cần thiết nhằm tránh hiện tượng nhấp nháy và tốn CPU, TaskGUI sử dụng cơ chế **change detection**:

```

1 // Chỉ vẽ lại khi có bất kỳ thay đổi nào
2 if (g_app.temp_x10 != last_temp_x10) {
3     last_temp_x10 = g_app.temp_x10;
4     need_redraw = true;

```

```
5 }
6 if (g_app.play_enabled != last_play_enabled) {
7     last_play_enabled = g_app.play_enabled;
8     need_redraw = true;
9 }
10 // Định kỳ 150ms để cập nhật đồng hồ RTC
11 if ((HAL_GetTick() - last_periodic) >= 150u) {
12     need_redraw = true;
13 }
14 if (need_redraw) ui_refresh_dynamic();
```

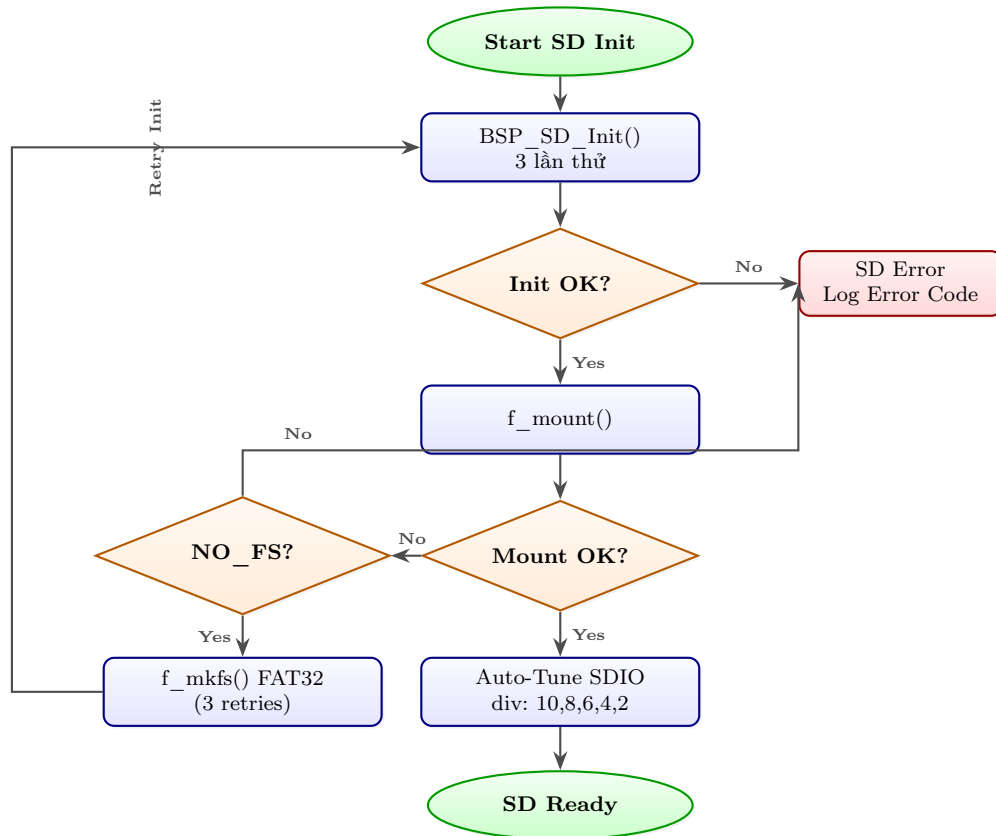
Listing 4.5: TaskGUI — Smart Redraw chỉ vẽ khi có thay đổi

4.3 TaskSD — Quản lý SD Card và ghi dữ liệu

4.3.1 Quy trình khởi tạo SD Card

TaskSD thực hiện quy trình khởi tạo nhiều bước với cơ chế retry robust:

1. **Init phần cứng:** Gọi `BSP_SD_Init()` được thiết lập lại với 3 lần thử, ưu tiên chế độ 1-bit.
2. **Mount filesystem:** Gọi `f_mount()` với FatFs.
3. **Auto-format:** Nếu phát hiện `FR_NO_FILESYSTEM`, khởi động lại SD rồi gọi `f_mkfs()` để định dạng FAT32 với tối đa 3 lần thử.
4. **Auto-tune:** Sau khi mount, thử tăng tốc clock SDIO qua các mức divider {10, 8, 6, 4, 2}, kiểm tra filesystem sau mỗi bước.



Hình 4.3: Flowchart quy trình khởi tạo SD Card

4.3.2 Override BSP_SD_Init

Hàm BSP_SD_Init() được override để cải thiện độ ổn định:

```

1 uint8_t BSP_SD_Init(void) {
2     for (uint32_t retry = 0; retry < 3; retry++) {
3         HAL_SD_DeInit(&hsd);
4         HAL_Delay(10);
5
6         if (HAL_SD_Init(&hsd) == HAL_OK) {
7             // Ưu tiên 1-bit mode để tăng do ổn định trên Open405R
8             if (HAL_SD_ConfigWideBusOperation(&hsd,
9                 SDIO_BUS_WIDE_1B) == HAL_OK)
10                return MSD_OK;
11            // Fallback: thu 4-bit
12            if (HAL_SD_ConfigWideBusOperation(&hsd,
13                SDIO_BUS_WIDE_4B) == HAL_OK)
14                return MSD_OK;
15        }
16        HAL_Delay(25);
17    }
18    return MSD_ERROR;
19 }
    
```

Listing 4.6: BSP_SD_Init override — Retry + fallback 1-bit/4-bit

4.3.3 Ghi dữ liệu mỗi 3 giây (dựa trên RTC)

Khi nhận SD_FLAG_PLAY_START, TaskSD bật chế độ logging. Thời gian được tính bằng giây RTC thay vì HAL_GetTick() để đảm bảo chính xác chu kỳ 3 giây:

```

1 if (is_logging && (elapsed_sec >= 3u)) {
2     last_log_sec = current_sec;
3
4     int16_t snap_temp = g_app.temp_x10;
5     uint16_t snap_adc = g_app.adc_raw_temp;
6     uint16_t snap_analog = g_app.adc_raw_analog;
7
8     fres = f_open(&log_file, "data.txt",
9                 FA_OPEN_APPEND | FA_WRITE);
10    if (fres == FR_OK) {
11        f_printf(&log_file,
12               "%02u:%02u:%02u|%02u/%02u/%04u|%d.%u C|T:%u|A:%u\n",
13               rtc_time.Hours, rtc_time.Minutes, rtc_time.Seconds,
14               rtc_date.Date, rtc_date.Month, 2000+rtc_date.Year,
15               snap_temp/10, abs(snap_temp)%10,
16               snap_adc, snap_analog);
17        f_sync(&log_file);
18        f_close(&log_file);
19    }
20 }
```

Listing 4.7: TaskSD — Ghi dữ liệu vào file

Định dạng file: Mỗi dòng có cấu trúc:

HH:MM:SS|DD/MM/YYYY|XX.X C|T:XXXX|A:XXXX

Ví dụ: 14:30:15|27/04/2026|25.3 C|T:0942|A:2048

Trong đó T: là ADC chip nhiệt độ, A: là ADC analog test board.

4.3.4 Đọc file cho chế độ xem dữ liệu

Khi nhận SD_FLAG_READ_REQUEST, TaskSD thực hiện:

1. **Pass 1:** Đọc file để đếm tổng số dòng `total_lines`.
2. **Tính phân trang:** `total_pages = ceil(total_lines / 6)`.
3. **Tính vị trí đọc ngược:** Trang 0 tương ứng với các dòng cuối file. Trang N tương ứng với các dòng đầu file.
4. **Pass 2:** Đọc file, nhảy đến vị trí cần, copy 6 dòng vào buffer.
5. **Đảo ngược:** Reverse buffer để dòng cuối cùng hiển thị lên đầu.
6. Đặt cờ `g_sd_viewer.ready = true` để TaskGUI vẽ.

4.3.5 Xóa dữ liệu

Khi nhận `SD_FLAG_DELETE`, `TaskSD`:

- Dừng logging (`is_logging = false`).
- Gọi `f_unlink("data.txt")` để xóa file.
- Reset viewer metadata và đặt `ready = true`.

4.4 Mã lỗi SD Card

Hệ thống sử dụng hệ thống mã lỗi riêng để debug qua LCD:

Bảng 4.2: Bảng mã lỗi SD Card hiển thị trên LCD

Mã	Ý nghĩa
CHK	Đang khởi tạo/kiểm tra
FMT	Đang format FAT32
RDY	SD sẵn sàng nhưng chưa mount
NOFS	Thẻ không có filesystem
OK	Hoạt động bình thường
ERR + 11	CMD Response Timeout
ERR + 12	CMD CRC Fail
ERR + 13	Data Timeout
ERR + 14	Data CRC Fail
ERR + 21	DMA Error
ERR + 200+	FatFs error code

Chương 5

Kết Quả Thực Nghiệm

5.1 Kết quả hiển thị LCD

5.1.1 Màn hình chính



Hình 5.1: Kết quả hiển thị giao diện Home trên LCD

Mô tả giao diện Home:

- **Header:** Gradient cam-hồng với tên nhóm “NHOM 2” và badge “LIVE” (viền cam, nền kem).

- **Card đo lường (2 cột):**

- Cột trái “Chip Temp”: Giá trị nhiệt độ XX.X°C với font lớn (FONT_1608), phần thập phân màu tím accent.
- Cột phải “ADC Values”: Hiển thị “Chip: XXXX” (ADC nhiệt độ) và “Test: XXXX” (ADC analog ngoại), phân cách bằng đường kẻ ngang.

- **Nút PLAY:** Gradient xanh lá khi active, trắng kem khi idle, viền bo góc đôi.

- **Nút PAUSE:** Gradient cam khi active, trắng kem khi idle.

- **Panel System:** Nền gradient xanh nhạt-tím nhạt, hiển thị: Play ON/OFF, Pause ON/OFF, SD status (CHK/OK/ERR/FMT/NOFS), tọa độ touch.

- **Footer:** Đồng hồ thời gian thực HH:MM:SS|DD/MM/YYYY, font nhỏ FONT_1206, căn giữa.

5.1.2 Màn hình khi đang ghi dữ liệu



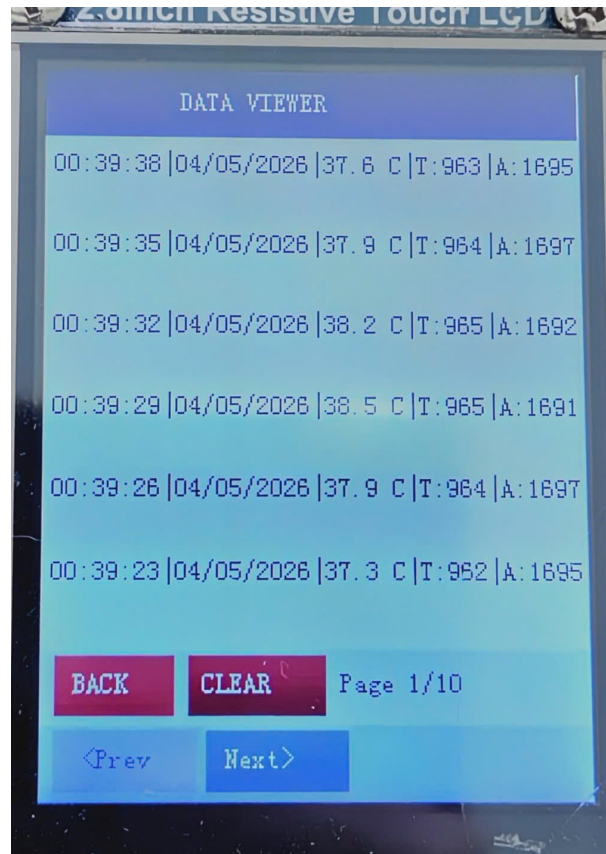
Hình 5.2: Kết quả hiển thị LCD khi đang ở chế độ PLAY

Khi nút PLAY được nhấn:

- Nút PLAY chuyển sang gradient xanh lá sáng, text trắng.
- Panel System hiển thị “Play: ON” (màu xanh lá).

- SD status hiện “OK” (hoặc mã lỗi nếu có vấn đề).
- Nhiệt độ và ADC tiếp tục cập nhật realtime.

5.1.3 Màn hình xem dữ liệu



Hình 5.3: Kết quả hiển thị giao diện xem dữ liệu thực tế trên LCD

Mô tả giao diện xem dữ liệu:

- **Header:** Gradient tím tối, text trắng “XEM DỮ LIỆU”.
- **Danh sách dữ liệu:** 6 dòng mỗi trang, với nền xen kẽ màu tím nhạt và trắng tím. Mỗi dòng hiển thị đầy đủ: Giờ|Ngày|Nhiệt độ|ADC Chip|ADC Analog.
- **Dữ liệu mới nhất lên đầu:** Trang 1 chứa các bản ghi mới nhất.
- **Nút BACK** (đỏ): Quay về Home.
- **Nút CLEAR** (đỏ đậm): Xóa toàn bộ file data.txt, chỉ hiển thị khi không ở chế độ ghi.
- **Nút Trước/Sau** (tím): Chuyển trang, vô hiệu hóa (màu xám) khi ở trang đầu hoặc trang cuối.
- **Thông tin trang:** “Page X/Y” hiển thị ở bên phải.
- Tất cả các nút điều hướng đặt ở **nửa trái** màn hình nhằm đảm bảo vùng an toàn (safe zone).

5.2 Kết quả dữ liệu trên SD Card

22:52:14	03/05/2026	32.4	C	T:945	A:2038
22:52:23	03/05/2026	32.7	C	T:946	A:2039
22:56:11	03/05/2026	32.7	C	T:947	A:2039
22:56:21	03/05/2026	33.0	C	T:947	A:2036
22:56:30	03/05/2026	33.0	C	T:949	A:2037
22:56:40	03/05/2026	33.0	C	T:947	A:2030
22:56:49	03/05/2026	33.3	C	T:948	A:2036
22:56:58	03/05/2026	33.0	C	T:948	A:2035
22:57:08	03/05/2026	32.7	C	T:947	A:2036
22:57:17	03/05/2026	33.0	C	T:948	A:2032
22:57:27	03/05/2026	33.6	C	T:949	A:2034
22:57:36	03/05/2026	33.0	C	T:948	A:2035
23:08:07	03/05/2026	32.7	C	T:948	A:2037
23:53:36	03/05/2026	35.8	C	T:957	A:2038
23:53:39	03/05/2026	36.4	C	T:957	A:2037
23:53:42	03/05/2026	35.8	C	T:957	A:2038
23:53:45	03/05/2026	35.5	C	T:955	A:2039
23:53:48	03/05/2026	35.5	C	T:956	A:2042
23:53:51	03/05/2026	36.4	C	T:959	A:2037
23:53:54	03/05/2026	35.2	C	T:955	A:2035
23:53:57	03/05/2026	35.5	C	T:956	A:2038

Hình 5.4: Nội dung file data.txt mở trên máy tính

Nhận xét:

- Mỗi dòng ghi đúng 3 giây một lần (dựa trên RTC).
- Nhiệt độ ổn định trong khoảng $\pm 0.2^{\circ}\text{C}$ giữa các lần đo liên tiếp nhờ sử dụng bộ lọc EMA.
- Giá trị ADC chip (T:) tương ứng với nhiệt độ tính được.
- Giá trị ADC analog (A:) phản ánh đúng điện áp trên PC0.

5.3 Phân tích hiệu năng RTOS

Bảng 5.1: Thống kê thời gian thực thi các Task

Task	Chu kỳ	Thời gian thực thi	CPU ước tính	Stack sử dụng
TaskSensor	20 ms	< 0.1 ms	< 0.5%	~200 B / 1 KB
TaskGUI (vẽ giao diện)	2 ms + 150 ms	~ 8–15 ms	~ 5–10%	~2 KB / 4 KB
TaskSD (ghi file)	3000 ms	~ 20–50 ms	< 2%	~3 KB / 4 KB
Tổng			< 15%	

Nhận xét hiệu năng:

- Tổng tải CPU ước tính < 15%, hệ thống có nhiều dư địa để mở rộng thêm task.
- TaskSensor ưu tiên cao nhất nên không bao giờ bị block bởi các task khác — đảm bảo cảm biến luôn được đọc đúng chu kỳ 20ms.
- TaskSD chỉ chiếm CPU khi ghi file (mỗi 3s), không ảnh hưởng đến độ mượt LCD.
- Idle task (được FreeRTOS tạo tự động) chiếm >85% thời gian CPU.

5.4 Kiểm thử chức năng

Bảng 5.2: Bảng kiểm thử chức năng hệ thống

STT	Chức năng kiểm thử	Kết quả	Ghi chú
1	Hiển thị nhiệt độ realtime	Đạt	Cập nhật mỗi 150ms
2	Hiển thị ADC chip và analog	Đạt	2 cột riêng biệt
3	Nhấn PLAY bắt đầu ghi	Đạt	Phản hồi <100ms
4	Ghi dữ liệu mỗi 3 giây	Đạt	Dựa trên RTC
5	Nhấn PAUSE xem dữ liệu	Đạt	Dữ liệu mới nhất lên đầu
6	Phân trang Trước/Sau	Đạt	6 dòng/trang
7	Xóa dữ liệu	Đạt	Chỉ cho phép khi không ghi
8	Thoát xem dữ liệu	Đạt	Vẽ lại Home hoàn chỉnh
9	Auto-format SD Card	Đạt	FAT32, tối đa 3 lần thử
10	Auto-tune tốc độ SDIO	Đạt	Tìm mức clock tối ưu
11	Đồng hồ RTC footer	Đạt	Cập nhật mỗi giây
12	Cảm ứng press-and-release	Đạt	Không kích hoạt nhầm
13	Rút/cắm lại SD Card	Đạt	Tự động re-init

Chương 6

Kết Luận

6.1 Kết quả đạt được

Đồ án đã hoàn thành đầy đủ các yêu cầu đề ra, xây dựng thành công hệ thống giám sát nhiệt độ thời gian thực trên nền tảng STM32F405 + FreeRTOS:

1. **Đọc nhiệt độ liên tục:** TaskSensor đọc cảm biến nhiệt độ nội bộ chip STM32 mỗi 20ms, áp dụng bộ lọc EMA cho cả 3 kênh ADC (Temp, VREFINT, IN10). Giá trị nhiệt độ được bù trừ sụt áp nguồn (VREFINT) và nội suy 2 điểm hiệu chuẩn nhà máy, hiển thị với 1 chữ số thập phân.
2. **Đọc ADC ngoại:** Kênh ADC1_IN10 (PC0) đọc thêm giá trị analog từ biến trở/cảm biến bên ngoài, hiển thị song song với nhiệt độ chip trên giao diện 2 cột.
3. **Ghi dữ liệu khi Play:** Khi người dùng chạm nút PLAY, TaskSD ghi bản ghi đầy đủ {Thời gian RTC, Nhiệt độ, ADC chip, ADC analog} vào file `data.txt` trên SD Card mỗi 3 giây (tính bằng RTC). Sử dụng FatFs với chế độ `FA_OPEN_APPEND`.
4. **Xem dữ liệu khi Pause:** Chế độ xem dữ liệu hiển thị lịch sử dữ liệu với phân trang 6 dòng mỗi trang, dữ liệu mới nhất lên đầu, hỗ trợ điều hướng Trước/Sau/Trở về/Xóa. Tất cả nút nằm ở nửa trái.

6.2 Các kỹ thuật nổi bật

- **RTOS Task Architecture:** 3 task với mức ưu tiên phân tầng (High → AboveNormal → Normal), đảm bảo cảm biến không bao giờ bị block. Tổng tải CPU <15%.
- **Thread Notification Flags + Phantom Trigger Protection:** Cơ chế giao tiếp inter-task zero-copy, kèm macro `SD_FLAGS_VALID()` kiểm tra bit MSB để loại mã lỗi CMSIS-RTOS.
- **Mutex SPI:** Bảo vệ bus SPI1 khi LCD và Touch Panel chia sẻ, kèm SPI recovery khi bị kẹt state.
- **VREFINT:** Bù trừ sụt áp nguồn trong phép đo ADC, tăng độ chính xác nhiệt độ.
- **EMA Filter 3 kênh:** Bộ lọc trung bình động cho Temp, VREFINT và ADC analog.

- **Touch Debounce 2 lớp:** Lọc median 3 điểm + kiểm tra stability span, kết hợp press-and-release.
- **SD Auto-Format + Auto-Tune:** Tự động format FAT32 nếu thẻ mới, tự động tìm tốc độ clock SDIO tối ưu.
- **BSP_SD_Init Override:** Retry 3 lần, ưu tiên 1-bit mode với fallback 4-bit.
- **Smart Redraw:** Chỉ vẽ lại LCD khi có thay đổi dữ liệu, giảm flicker và tiết kiệm CPU.
- **Modern Card UI:** Giao diện gradient, bo góc, bảng màu ấm, badge “LIVE”, footer RTC.
- **Xem dữ liệu đảo ngược:** Dữ liệu mới nhất hiển thị lên đầu trang 1.
- **RTC-based Timing:** Ghi dữ liệu theo giây RTC (không dùng HAL_GetTick) để đảm bảo chính xác.

6.3 Hạn chế

- Cảm biến nhiệt độ nội bộ có độ chính xác giới hạn ($\pm 1.5^{\circ}\text{C}$ theo datasheet), đo nhiệt độ nhân chip thay vì nhiệt độ môi trường, cần offset hiệu chuẩn.
- Một góc màn hình cảm ứng bị liệt (góc phải dưới), buộc phải thiết kế UI dồn nút sang nửa trái. Đây là lỗi phần cứng của module LCD cụ thể.
- RTC reset về thời gian biên dịch mỗi lần nạp firmware hoặc mất nguồn do không có pin backup CR2032.
- Giao diện LCD bị giới hạn bởi tốc độ SPI và khả năng vẽ pixel-by-pixel: không thể tạo animation phức tạp.
- File data.txt chỉ hỗ trợ append, không có cơ chế rotation (tự xóa dữ liệu cũ khi đầy).

6.4 Hướng phát triển

1. **Cảm biến ngoại:** Thêm cảm biến nhiệt độ bên ngoài như DS18B20 1-Wire, LM35 analog, hoặc BME280 I2C để đo nhiệt độ môi trường chính xác hơn và so sánh với nhiệt độ chip.
2. **Truyền dữ liệu lên máy tính:** Sử dụng USB CDC hoặc UART để truyền dữ liệu realtime lên PC, vẽ biểu đồ bằng Python/MATLAB.
3. **Kết nối IoT:** Thêm module WiFi như ESP8266 hoặc ESP32 qua UART để đẩy dữ liệu lên cloud như ThingSpeak, Firebase hoặc MQTT, cho phép giám sát từ xa.
4. **Biểu đồ trên LCD:** Hiển thị biểu đồ nhiệt độ theo thời gian trực tiếp trên LCD, lưu history 100–200 mẫu trong SRAM.

5. **Pin backup RTC:** Thêm pin CR2032 qua mạch sạc đơn giản để giữ thời gian khi mất nguồn chính.
6. **Cảnh báo ngưỡng:** Đặt ngưỡng nhiệt độ cao/thấp, kích hoạt buzzer hoặc LED cảnh báo khi vượt ngưỡng.
7. **File rotation:** Tự động tạo file mới theo số thứ tự khi file hiện tại vượt quá kích thước giới hạn.
8. **Chế độ hiệu chuẩn:** Thêm chế độ hiệu chuẩn cảm ứng trên LCD bằng cách chạm 3 điểm thay vì dùng hằng số cố định.

Tài liệu tham khảo

- [1] STMicroelectronics, “RM0090 Reference Manual – STM32F405/415, STM32F407/417, STM32F427/437, STM32F429/439,” Rev 21, 2023.
- [2] STMicroelectronics, “DS8626 Datasheet – STM32F405xx STM32F407xx,” Rev 10, 2020.
- [3] STMicroelectronics, “AN3964 Application Note – STM32F4 Internal Temperature Sensor,” Rev 1.
- [4] Richard Barry, “Mastering the FreeRTOS Real Time Kernel – A Hands On Tutorial Guide,” 2016.
- [5] FreeRTOS, “FreeRTOS API Reference,” <https://www.freertos.org/a00106.html>.
- [6] ChaN, “FatFs – Generic FAT Filesystem Module,” http://elm-chan.org/fsw/ff/00index_e.html.
- [7] ARM Ltd., “CMSIS-RTOS2 API Documentation,” https://arm-software.github.io/CMSIS_5/RTOS2/html/index.html.
- [8] Waveshare, “Open405R-C User Manual,” 2023, <https://www.waveshare.com/wiki/Open405R-C>.
- [9] Ilitek, “ILI9341 a-Si TFT LCD Single Chip Driver – Datasheet,” V1.11, 2011.
- [10] Shenzhen Xptek, “XPT2046 Touch Screen Controller Datasheet,” Rev 1.0.
- [11] STMicroelectronics, “UM1725 User Manual – Description of STM32F4 HAL and LL Drivers,” Rev 8, 2022.
- [12] STMicroelectronics, “STM32CubeIDE User Guide,” UM2609, Rev 6, 2023.